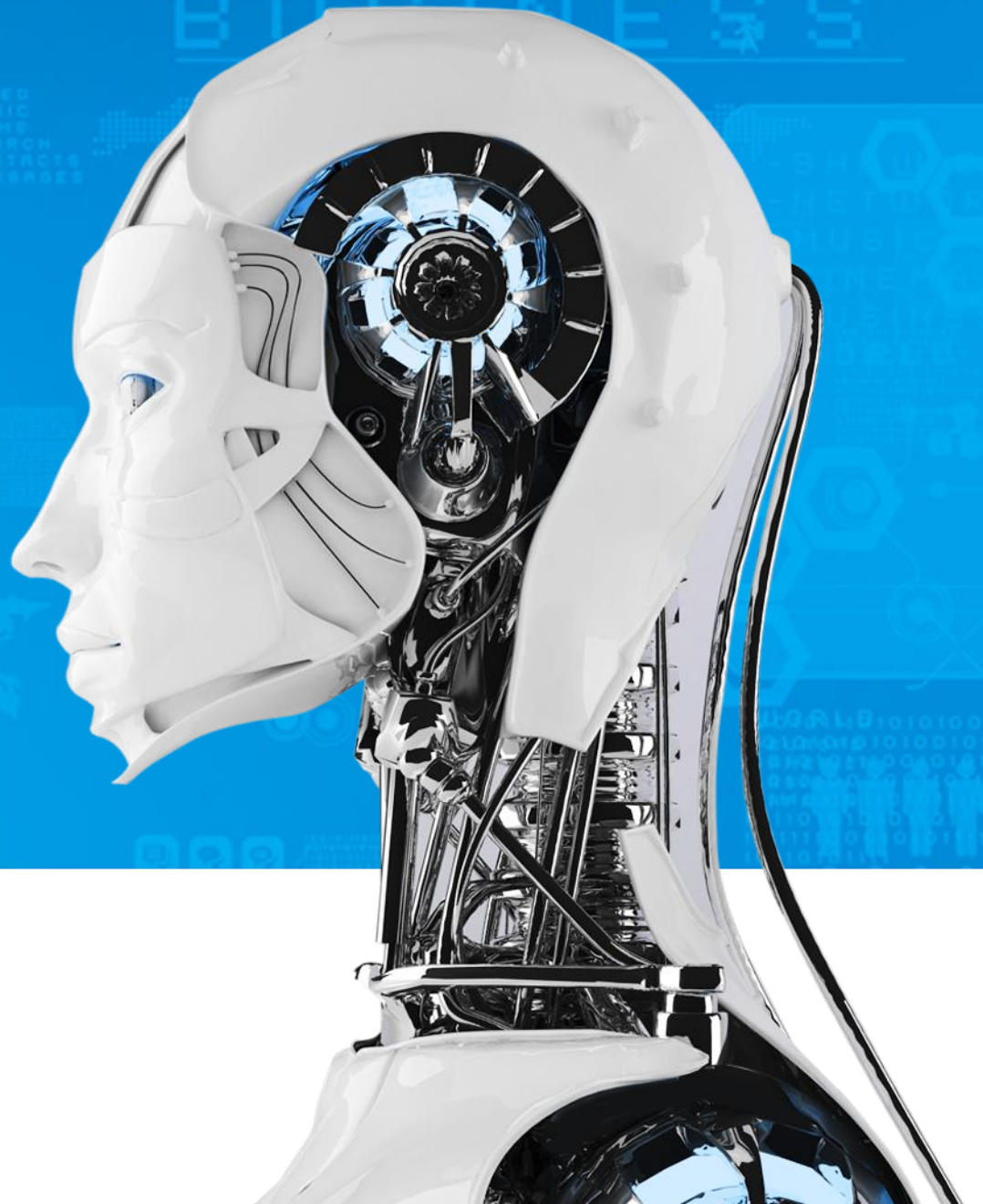
데이터 분석을 위한

NumPy



CONTENTS

01 NUMPY에 대한 이해

02 NUMPY 실습

03 (참고) MNIST 데이터 셋

04 MNIST 실습

- 파이썬의 대표적인 라이브러리 중 하나인 NumPy를 설명할 수 있다.
- NumPy의 ndarray 객체에 대해 설명할 수 있다.
- ndarray 객체를 생성하고, ndarray 객체의 함수와 변수를 활용할 수 있다.
- MNIST 데이터 셋에 대해 설명할 수 있다.

NumPy 소개

NumPy
(Numerical Python)

고성능의 수치 계산을 위해 만들어진
Python의 핵심 라이브러리

NumPy 소개

NumPy의 특징

1. C언어로 구현된 파이썬 라이브러리

2. Python 데이터 타입을 C의 int, float와 같은 데이터 포맷으로 변환하고 연산을 수행하여 속도 개선
   - Python은 객체지향언어임
   - int, float와 같은 데이터 타입도 객체로 처리해 다소 속도가 저하됨

3. ndarray(N-Dimesional Array) 객체 사용
   - 다차원배열을 효과적으로 다룰 수 있음
   - ndarray는 Tensorflow Tensor와 Pandas DataFrame 기반임

NumPy의 특징

4 벡터화 연산(Vectorized Operation) 지원

- 반복문을 사용하지 않고 배열의 모든 원소에 대해 반복 연산이 가능함

5 브로드캐스팅(Broadcasting) 지원

- 배열 간 연산 수행 시 배열의 크기가 다르더라도 크기가 작은 배열을 자동으로 반복 확장하여 크기가 큰 배열에 맞추어 연산을 수행함

ndarray의 구조

ndarray 객체

다차원배열을 효과적으로 다루기 위해 사용됨

멤버 변수

- 차원 (dim)
- 자료형 (dtype)
- 모양 (shape)
- 크기 (size)

ndarray의 구조

NumPy 사용법

- numpy 설치
  - ✓ pip install numpy(Anaconda 기본 설치)
- numpy 사용
  - ✓ import numpy as np

ndarray의 구조

ndarray 사용법

- ndarray 생성

```
nd1 = np.array([1, 2, 3, 4])
nd2 = np.array([[1, 2, 3], [4, 5, 6]])
nd3 = np.array([[[1, 2, 3], [4, 5, 6], [5, 6, 7], [10, 11, 12]],
                [[13, 14, 15], [16, 17, 18], [19, 20, 21],
                [22, 23, 24]]])
```

- ndarray 확인

```
type(nd1)
```

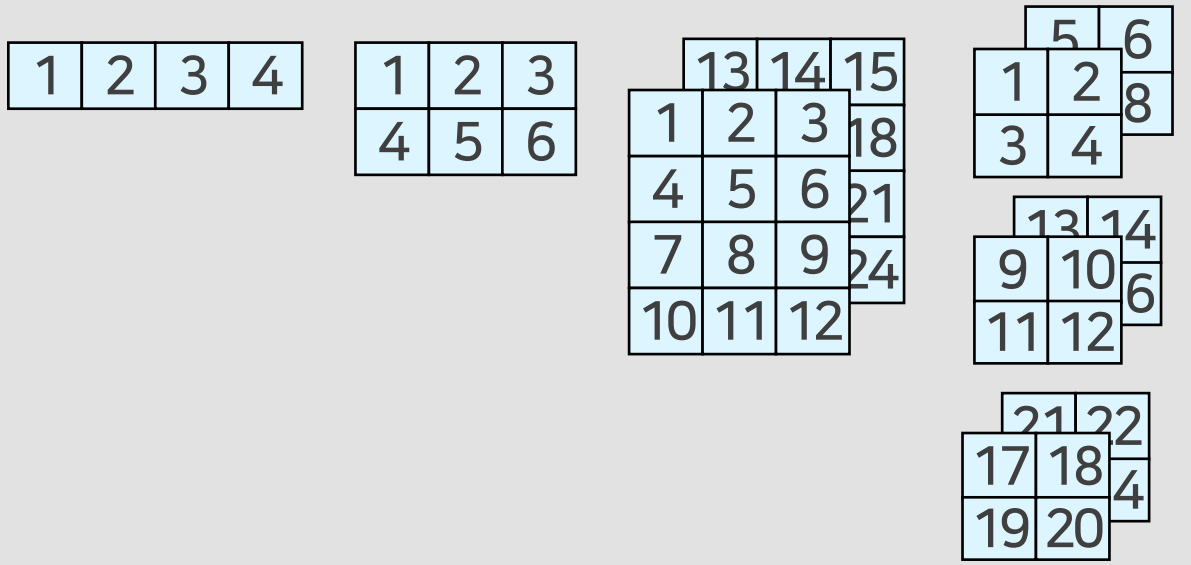
✓ numpy.ndarray

ndarray의 차원

1D, 2D, 3D, 4D... ND 등이며,
일반적으로 1D~4D 까지 많이 사용

ndarray.ndim 변수를 이용하여 ndarray의 차원 확인 가능

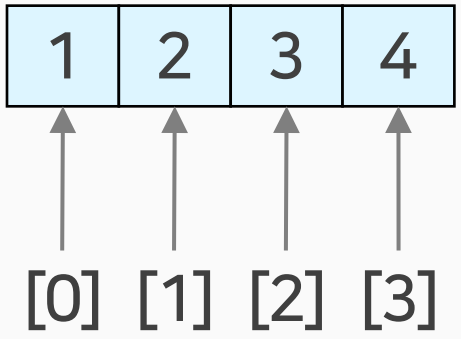
1차원(1D) 2차원(2D) 3차원(3D) 4차원(4D)



… N차원(ND)

ndarray의 차원

1차원(1D) ndarray




```
nd1 = np.array([1, 2, 3, 4])
```

```
print(nd1)
```

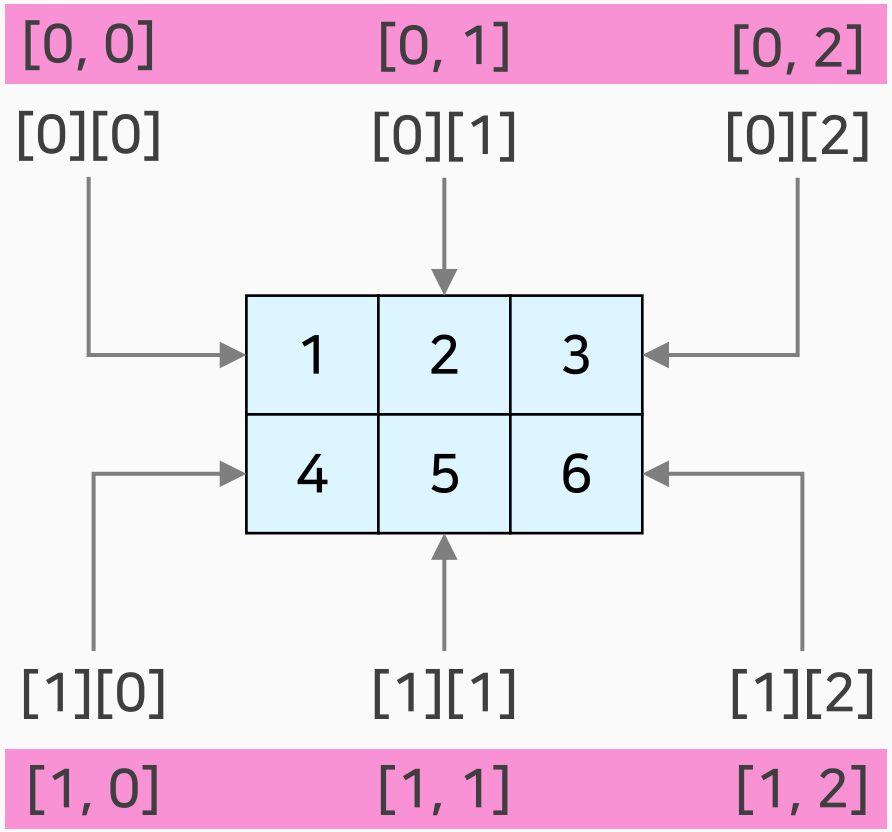
```
[1 2 3 4]
```

```
nd1.ndim
```

```
1
```

ndarray의 차원

2차원(2D) ndarray



ndarray의 차원

2차원(2D) ndarray

```
nd2 = np.array( [[ 1, 2, 3], [4, 5, 6]])
```

```
print(nd2)
```

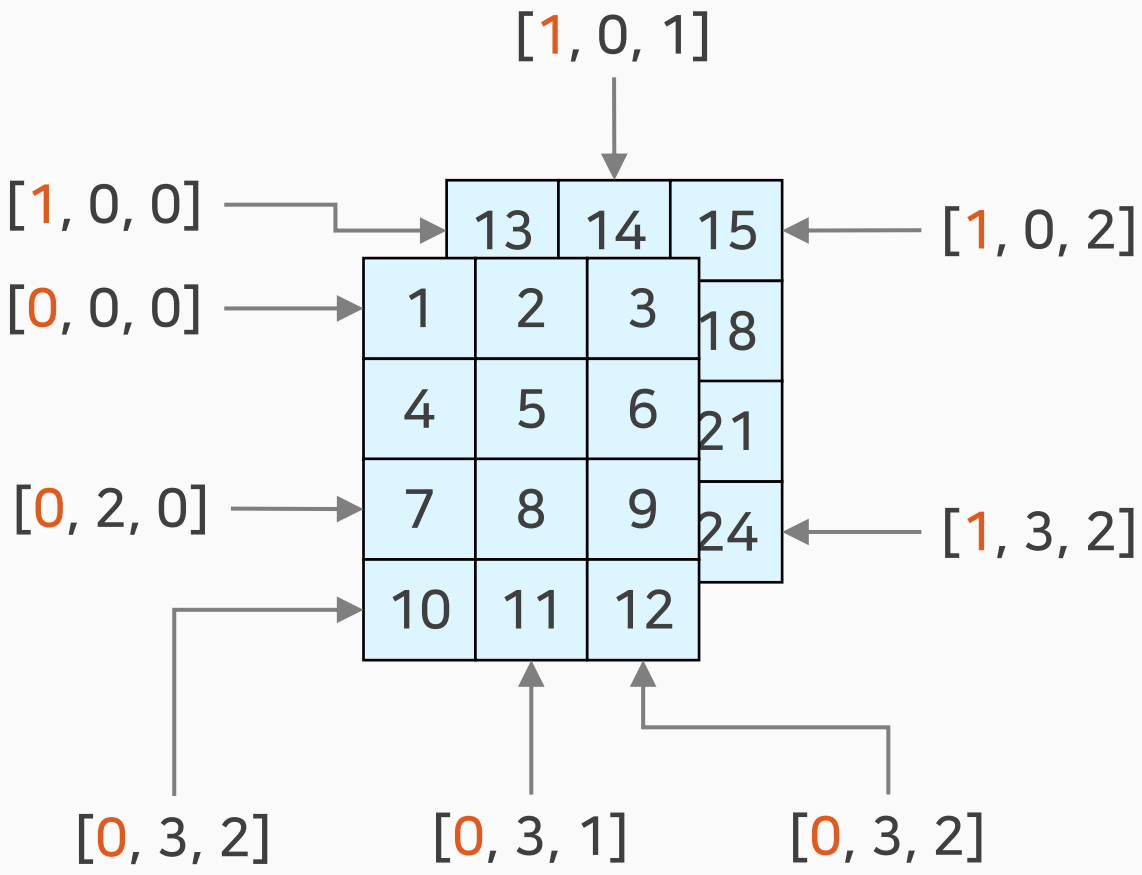
```
[[ 1 2 3]
 [ 4 5 6]]
```

```
nd2.ndim
```

```
2
```

ndarray의 차원

3차원(3D) ndarray



ndarray의 차원

3차원(3D) ndarray

```
nd3 = np.array([[[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]],
                [[13, 14, 15], [16, 17, 18], [19, 20, 21],
                [22, 23, 24]]])
```

```
print(nd3)
```

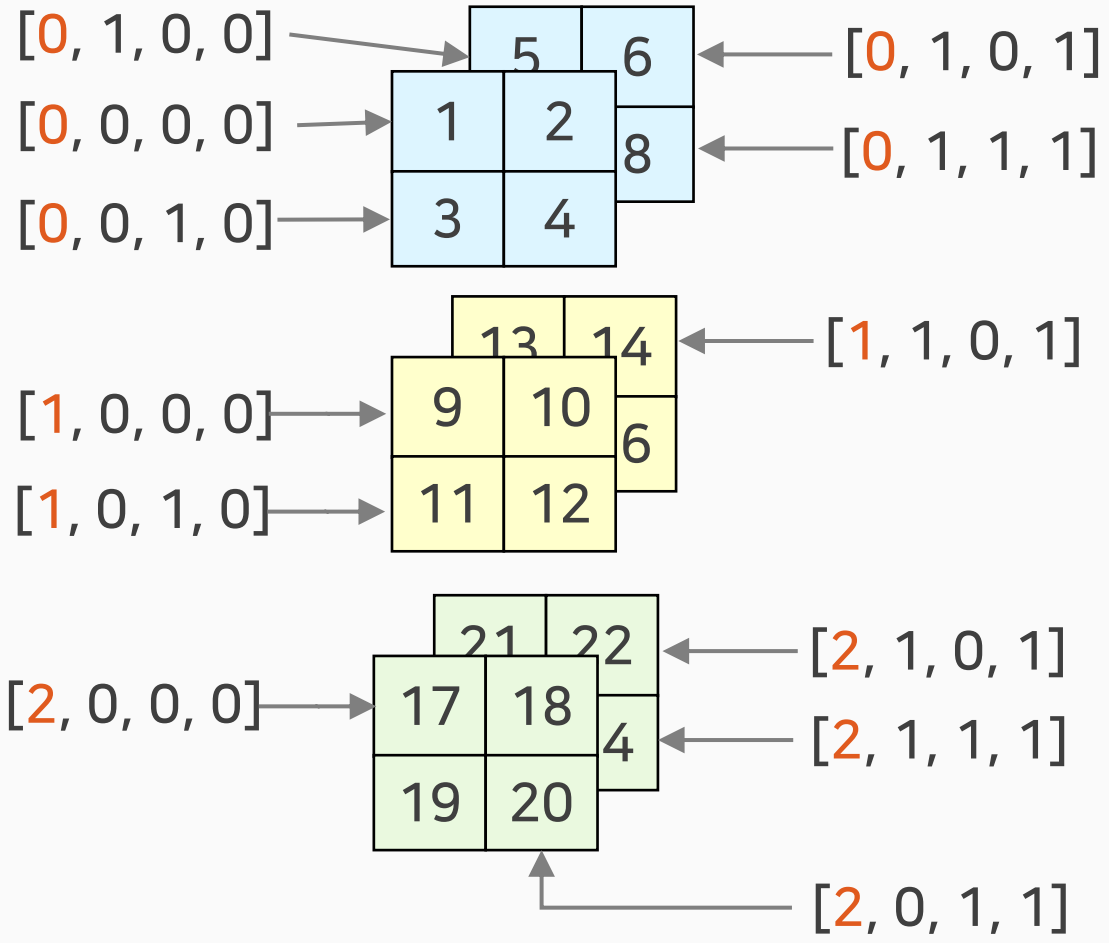
```
[[[ 1 2 3]
  [ 4 5 6]
  [ 7 8 9]
  [10 11 12]]
 [[ 13 14 15]
        ⋮
  [22 23 24]]]
```

```
nd3.ndim
```

```
3
```

ndarray의 차원

4차원(4D) ndarray



ndarray의 차원

4차원(4D) ndarray

```
nd4 = np.array([[[[1, 2], [3, 4], [5, 6], [7, 8]]],
                [[[9, 10], [11, 12], [13, 14], [15, 16]]],
                [[[17, 18], [19, 20], [21, 22], [23, 24]]]])
```

```
print(nd4)
```

```
[[[[ 1 2]
   [ 3 4]]

   [[ 5 6]
    [ 7 8]]]
      ⋮
   [[ 21 22]
    [ 23 23]]]
```

```
nd4.ndim
```

```
4
```

ndarray의 자료형

숫자형

- Boolean
- integer
- unsigned integer
- floating-point
- complex floating-point

날짜형

- datatime
- timedelata

문자형

- string
- unicode

객체형

- object

! numpy의 ndarray는 ndarray 객체 당 하나의 자료형만 지정 가능

ndarray의 자료형

숫자형 | 날짜형 | 문자형 | 객체형

Boolean	bool	Boolean(True, False)
integer	int8	8bit 정수형 8bit
	int16	16bit 정수형
	int32	32bit 정수형 32bit
	int64	64bit 정수형
unsigned integer	uint8	8bit 양의 정수형
	uint16	16bit 양의 정수형
	uint32	32bit 양의 정수형
	uint64	64bit 양의 정수형

ndarray의 자료형

숫자형 | 날짜형 | 문자형 | 객체형

floating-point	float16	16bit 부동 소수형
	float32	32bit 부동 소수형
	float64	64bit 부동 소수형
complex floating-point	complex64	64bit 복소수형(실수+허수)
	complex128	128bit 복소수형(실수+허수)

ndarray의 자료형

숫자형 | 날짜형 | 문자형 | 객체형

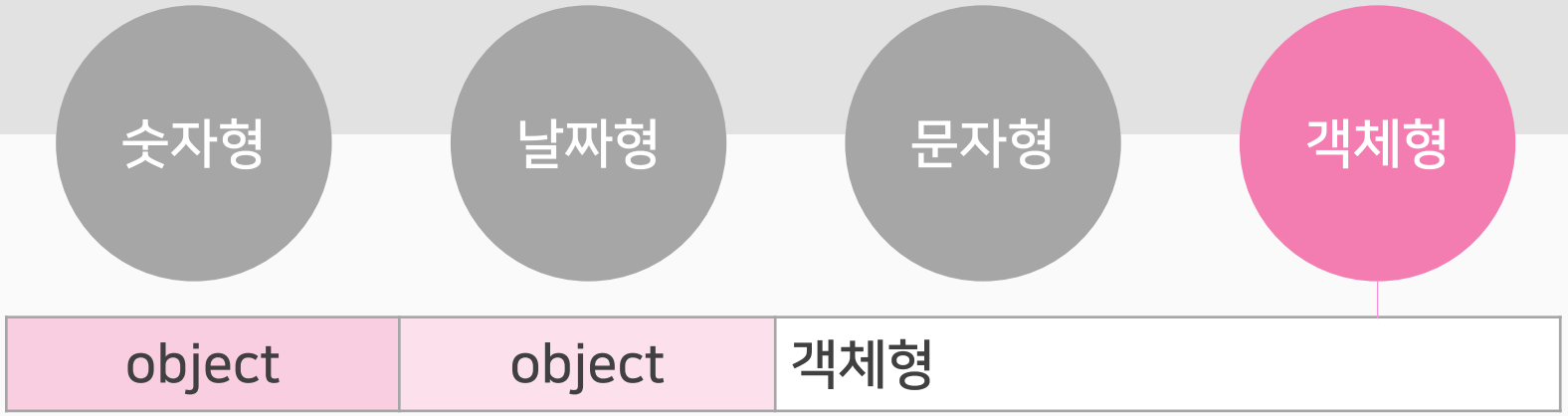
datetime	datetime64	64bit 날짜시간형
timedelata	timedelata64	64bit 날짜차이형

ndarray의 자료형

숫자형 | 날짜형 | 문자형 | 객체형

string	string_	Byte 문자열형
Unicode	str_	유니코드 문자열형

ndarray의 자료형



ndarray의 자료형

자료형 함수 사용 예시

1. 자료형 할당
   - nd1 = np.array([1.0, 2.1, 3.0, 4.2]) ← float64로 default 할당
   - nd2 = np.array([[1, 2, 3], [4, 5, 6]]) ← int32로 default 할당
   - nd2 = np.array([[1, 2, 3], [4, 5, 6]], dtype = np.float32)
   - nd2 = np.array([[1, 2, 3], [4, 5, 6]], dtype = 'float64')
2. ndarray 자료형 확인
   - print(nd1.dtype)
3. ndarray 자료형 형변환
   - nd1 = nd1.astype(np.int32)

ndarray의 모양과 크기

ndarray의 모양(Shape)

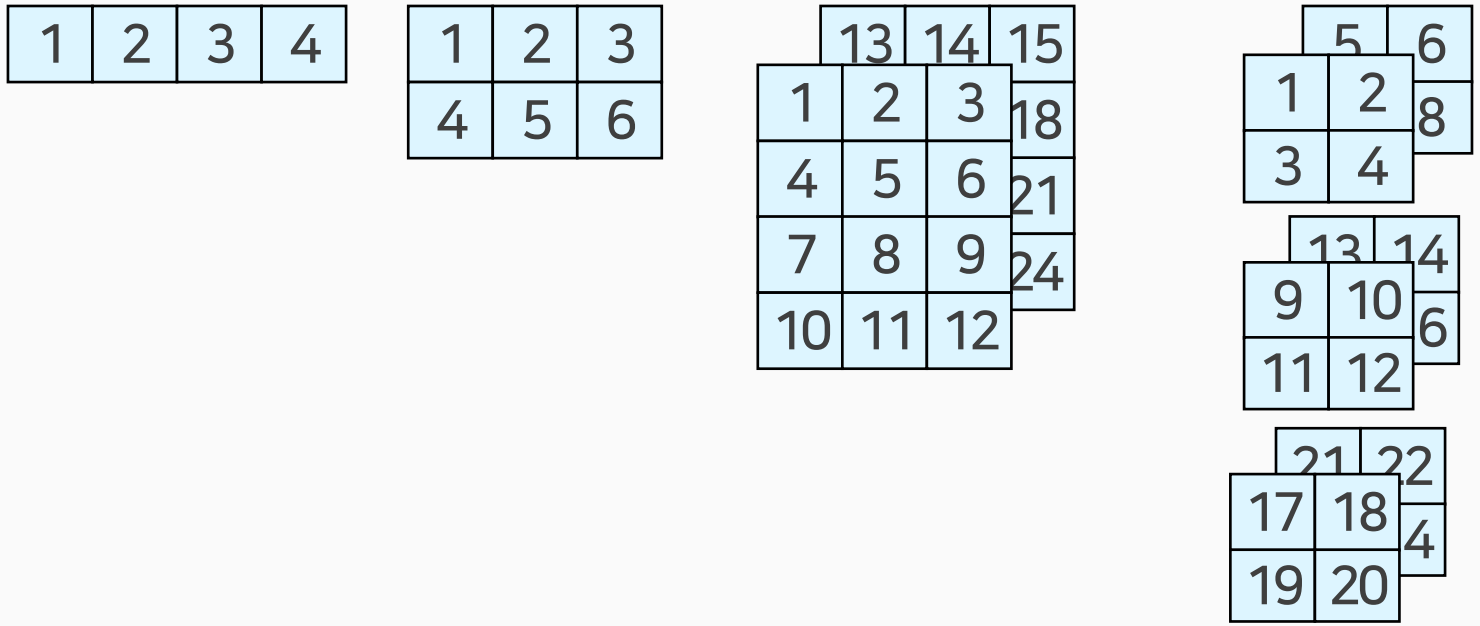
머신러닝/딥러닝에서 입력데이터의 shape를
맞추고 확인하는 과정은 매우 중요함

- ndarray.shape 변수를 이용하여 shape의 확인이 가능함
- 결과는 python 튜플 형태로 반환함

ndarray의 모양과 크기

ndarray의 모양(Shape)

1차원(1D)	2차원(2D)	3차원(3D)	4차원(4D)
shape : (4,)	shape : (2, 3)	shape : (2, 4, 3)	shape : (3, 2, 2, 2)



ndarray의 모양과 크기

ndarray의 크기(Size)

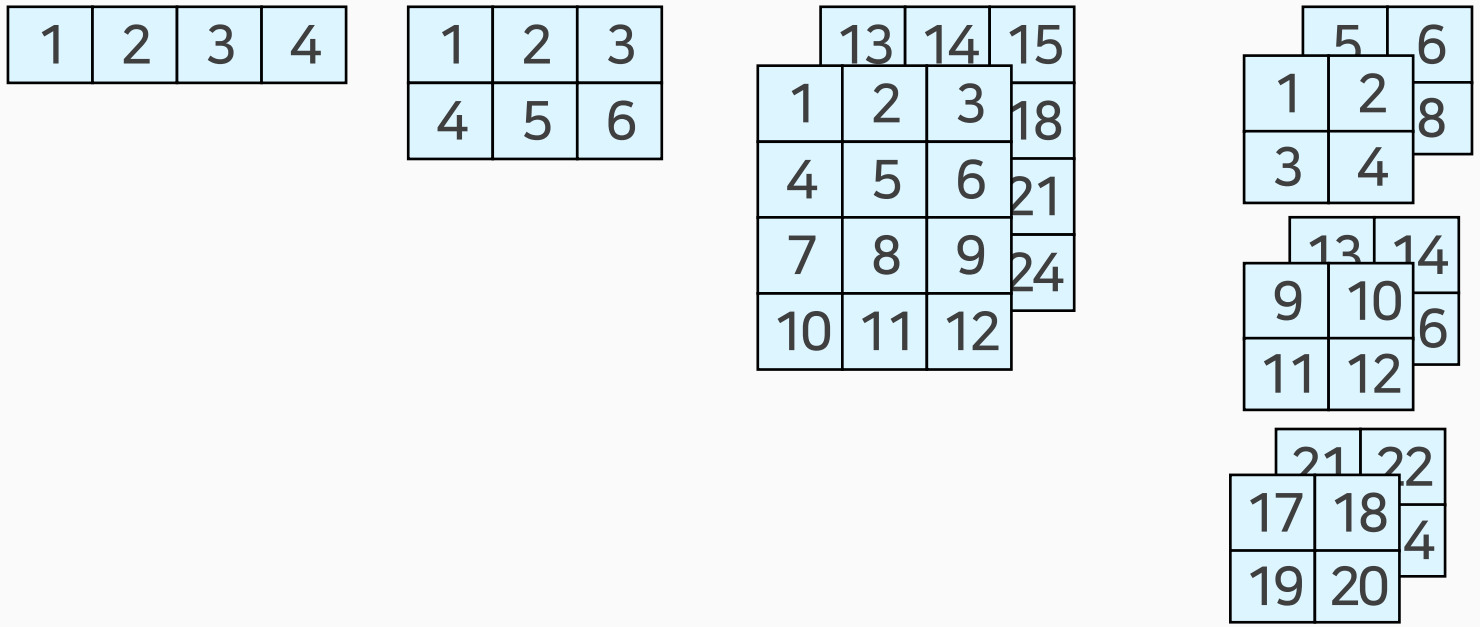
ndarray의 전체 크기(size)

- ndarray.size 변수를 이용하여 확인 가능
- 모든 Element의 개수
- shape의 모든 값들을 곱한 값

ndarray의 모양과 크기

ndarray의 크기(Size)

1차원(1D)	2차원(2D)	3차원(3D)	4차원(4D)
shape : (4,)	shape : (2, 3)	shape : (2, 4, 3)	shape : (3, 2, 2, 2)
size : 4(4*1)	size: 6(2*3)	size: 24(2*4*3)	size: 24(3*2*2*2)



ndarray의 모양과 크기

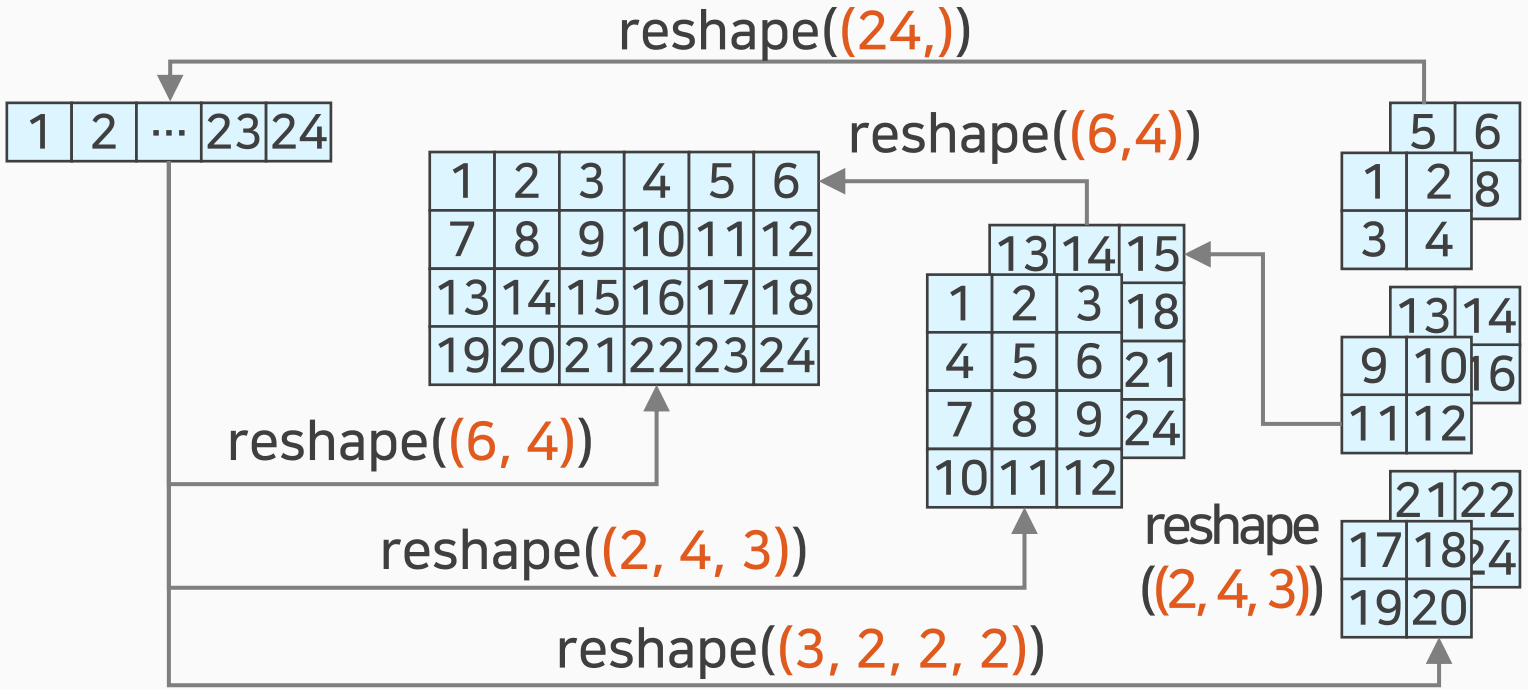
ndarray의 모양의 변환(Reshape)

- 입력데이터의 shape이 불일치한 경우, 이를 맞추기 위해 필요한 과정
- ndarray.reshape 변수를 이용하여 shape의 변환 가능
- 파라미터는 python tuple 형태
- 동일 차원내 변환 및 이종 차원 간 변환 가능
- ndarray의 size가 동일한 경우에만 reshape 가능

ndarray의 모양과 크기

ndarray의 크기(Size)

1차원(1D)	2차원(2D)	3차원(3D)	4차원(4D)
shape (24,)	shape (6, 4)	shape (2, 4, 3)	Shape (3, 2, 2, 2)



ndarray의 기타 함수

np.zeros(shape)

shape의 모양대로 0으로 채운 ndarray 생성

```
np.zeros((4, 3))
```

```
array([[0.,0.,0],
       [0.,0.,0],
       [0.,0.,0],
       [0.,0.,0],])
```

ndarray의 기타 함수

np.ones(shape)

shape의 모양대로 1로 채운 ndarray 생성

```
np.ones((3, 4))
```

```
array([[1.,1.,1.,1.],
       [1.,1.,1.,1.],
       [1.,1.,1.,1.]])
```

ndarray의 기타 함수

np.arrange(start, stop, step)

python의 range함수와 동일

ndarray의 기타 함수

np.arrange(start, stop, step)

```
np.arrange(12)
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11])
```

```
np.arrange(1, 10, 2)
```

```
array([1, 3, 5, 7, 9])
```

```
np.arrange(0, 10, 2)
```

```
array([0, 2, 4, 6, 8])
```

```
np.arrange(12).reshape(4, 3)
```

```
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8],
       [9, 10, 11]])
```

ndarray의 벡터화 연산과 브로드캐스팅

벡터화 연산(Vectorized Operation)

- 별도의 반복문(For Loop)을 사용하지 않음
- Element-Wise(같은 위치의 값 끼리)한 반복 연산이 가능함

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} + \begin{bmatrix} 2 & 2 \\ 2 & 2 \\ 2 & 2 \end{bmatrix} = \begin{bmatrix} 3 & 4 \\ 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} - \begin{bmatrix} 2 & 2 \\ 2 & 2 \\ 2 & 2 \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} * \begin{bmatrix} 2 & 2 \\ 2 & 2 \\ 2 & 2 \end{bmatrix} = \begin{bmatrix} 2 & 4 \\ 6 & 8 \\ 10 & 12 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} / \begin{bmatrix} 2 & 2 \\ 2 & 2 \\ 2 & 2 \end{bmatrix} = \begin{bmatrix} 0.5 & 1 \\ 1.5 & 2 \\ 2.5 & 3 \end{bmatrix}$$

ndarray의 벡터화 연산과 브로드캐스팅

브로드캐스팅
(Broadcasting)

딥러닝 학습을 위해 많이 사용되는
데이터 셋

ndarray의 벡터화 연산과 브로드캐스팅

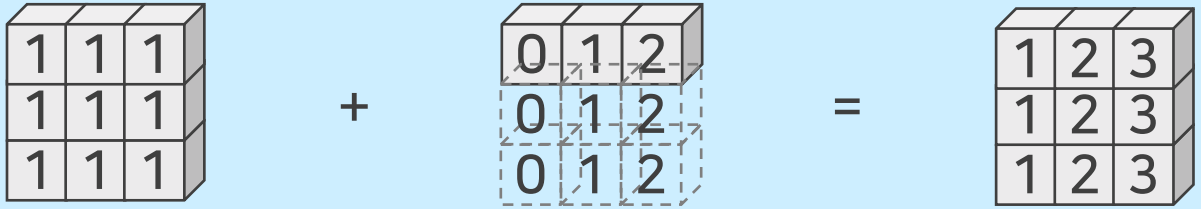
연산이 가능한 형태로
자동 Reshape

→

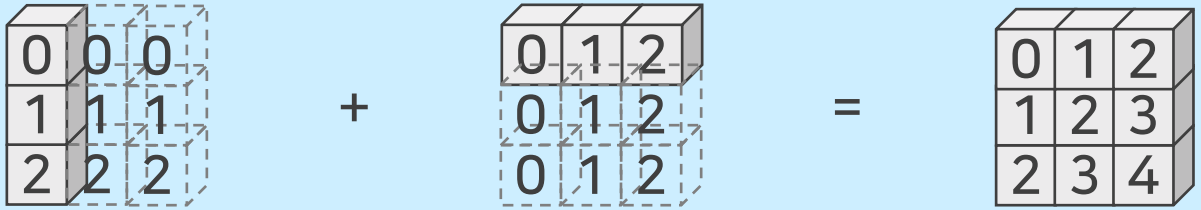
반복된 값을 자동으로
할당하여 연산 수행

1 np.arrage(3)+5

2 np.ones((3, 3))+np.arrange(3)



3 np.arange(3).reshape((3,1))+np.arrange(3)



<출처 : http://www.astroml.org/book_figures/appendix/fig_broadcast_visual.html>

(참고) MNIST 데이터 셋

딥러닝 데이터 셋
(Deep Learning Data Set)

딥러닝 학습을 위해 많이 사용되는
데이터 셋

(참고) MNIST 데이터 셋 개요

딥러닝
데이터 셋

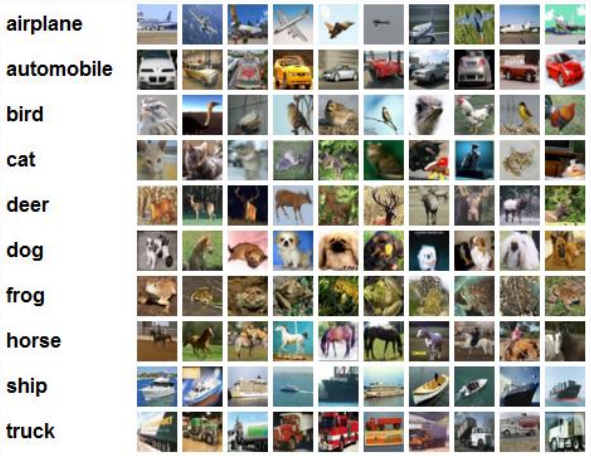
- MNIST 데이터 셋

- Fashion MNIST 데이터 셋

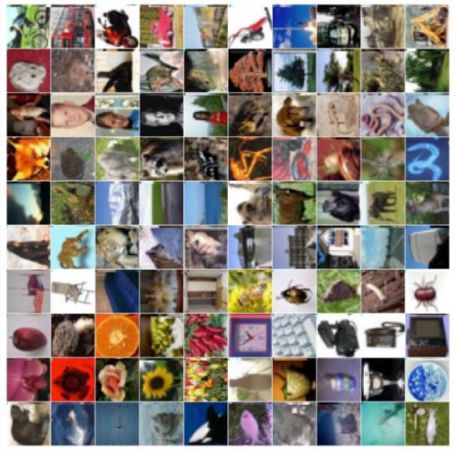
(참고) MNIST 데이터 셋 개요

딥러닝 데이터 셋

- CIFAR-10 데이터 셋



- CIFAR-100 데이터 셋

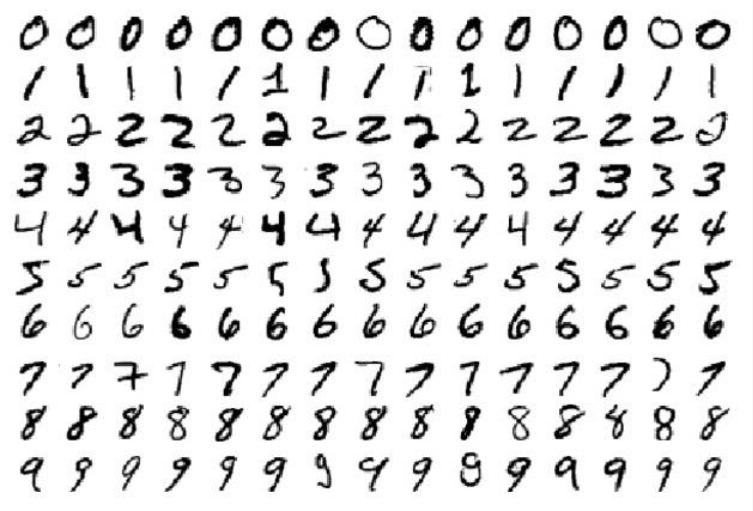


(참고) MNIST 데이터 셋 개요

MNIST 데이터 셋

컴퓨터 비전(Vision)분야에서
머신러닝/딥러닝 학습·예측을 위해 많이 사용되는 데이터 셋

- 인공지능 연구의 권위자 LeCun교수가 개발함
- Tensorflow, Keras 등 대부분의 프레임워크에서 기본 함수로 제공됨



(참고) MNIST 데이터 셋 개요

MNIST 데이터 셋의 특징

0~9까지의 수를 28 * 28픽셀 크기의 흑백이미지로 구성

ndarray type의 mnist data

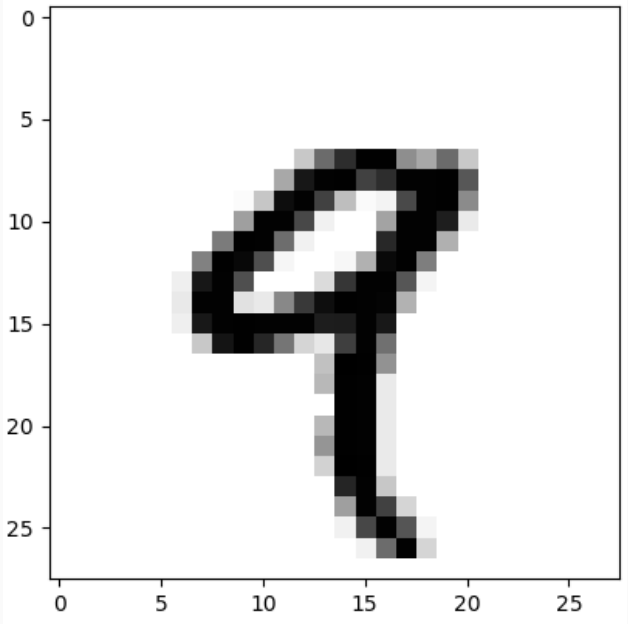
```
[[  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0  55 148 210 253 253 113  87 148  55   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0  87 232 252 253 189 210 252 252 253 168   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   4  57 242 252 190  65   5  12 182 252 253 116   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0  96 252 252 183  14   0   0  92 252 252 225  21   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0 132 253 252 146  14   0   0   0 215 252 252  79   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0 126 253 247 176   9   0   0   8  78 245 253 129   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0  16 232 252 176   0   0   0  36 201 252 252 169  11   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0  22 252 252  30  22 119 197 241 253 252 251  77   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0  16 231 252 253 252 252 252 226 227 252 231   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0  55 235 253 217 138  42  24 192 252 143   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0  62 255 253 109   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0  71 253 252  21   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0 253 252  21   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0  71 253 252  21   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0 106 253 252  21   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0  45 255 253  21   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0 218 252  56   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0  96 252 189  42   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0  14 184 252 170  11   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0  14 147 252  42   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]]
```

(참고) MNIST 데이터 셋 개요

MNIST 데이터 셋의 특징

0~9까지의 수를 28 * 28픽셀 크기의 흑백이미지로 구성

ndarray의 이미지 표현



(참고) MNIST 데이터 셋 개요

MNIST 데이터 셋의 Pair

Training 데이터 셋

- train_images: 60,000개의 Training Data 이미지
- train_labels: 60,000개의 Training Data에 대한 Label

Test 데이터 셋

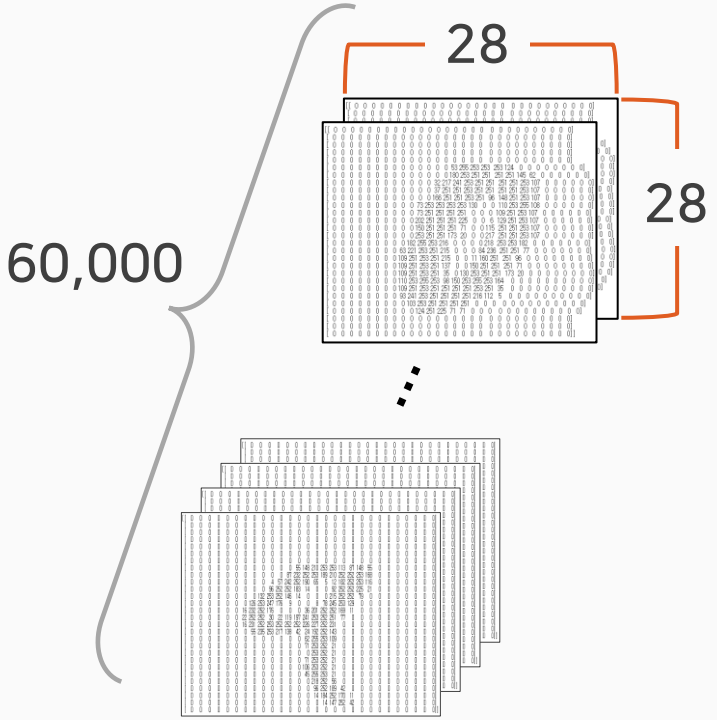
- test_images: 10,000개의 Training Data 이미지
- test_labels: 10,000개의 Training Data에 대한 Label

(참고) MNIST Training 데이터 셋

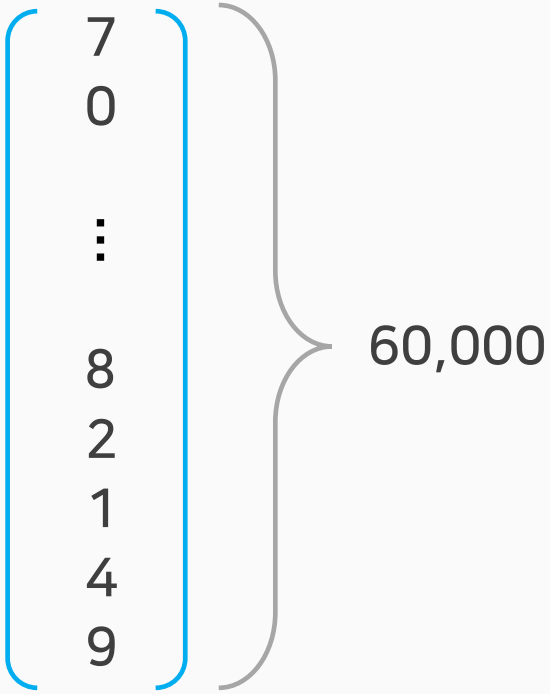
Shape(60000)의 경우

MNIST training_images
shape : (60000, 28, 28)

MNIST training_labels
shape : (60000,)

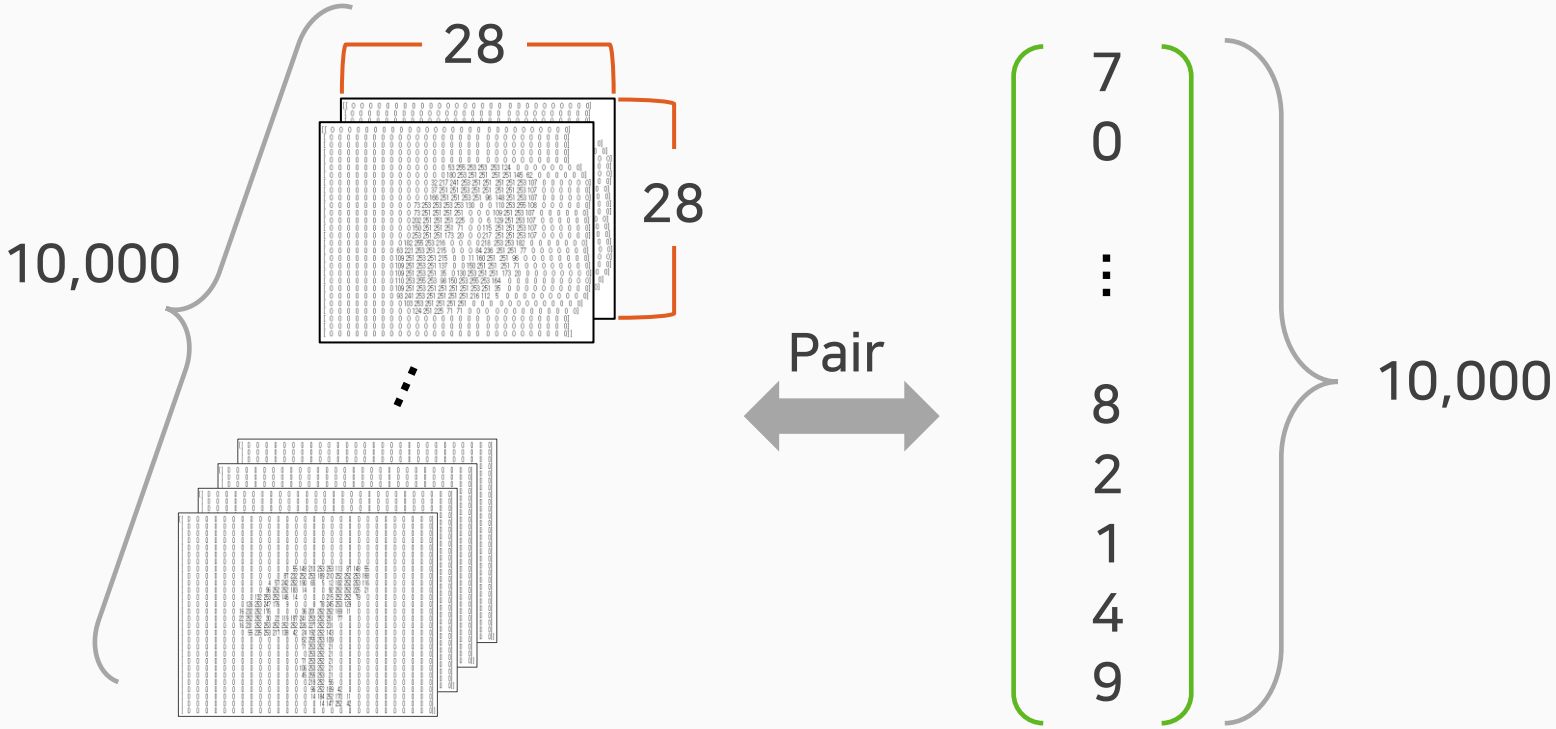


Pair



(참고) MNIST Training 데이터 셋

Shape(10000)의 경우

MNIST test_images
shape : (10000, 28, 28)

MNIST test_labels
shape : (10000,)